SPPU TE BE COMP Click to join telegram

Tree definitions & terminology.

Tree: Definitions & Terminology (DSA)

Definition of Tree

A **tree** is a **non-linear hierarchical data structure** consisting of a **finite set of nodes** connected by edges, with:

- One **root node**
- **Zero or more subtrees**
- **No cycles**

Basic Terminology

1. Node

An individual element of a tree that contains **data** and links to other nodes.

---

2. Root

The **topmost node** of the tree.

- Has **no parent**

---

3. Edge

A **connection** between two nodes.

---

4. Parent

A node that has **one or more children**.

SPPU TE BE COMP Click to join telegram

5. Child

A node that is a **direct descendant** of another node.

---

6. Siblings

Nodes that share the **same parent**.

---

7. Leaf (External Node)

A node with **no children**.

---

8. Internal Node

A node with **at least one child**.

---

9. Degree of a Node

The **number of children** of that node.

---

10. Degree of a Tree

The **maximum degree** of any node in the tree.

---

11. Level of a Node

The **distance from the root**.

- Root is at **level 0** (sometimes level 1, depending on convention)

---

12. Depth of a Node

The **number of edges from the root** to that node.

SPPU TE BE COMP Click to join telegram

13. Height of a Node

The **number of edges on the longest downward path** from that node to a leaf.

---

14. Height of a Tree

The **height of the root node**.

---

15. Path

A **sequence of nodes** connected by edges.

---

16. Path Length

The **number of edges** in a path.

---

17. Ancestor

Any node on the path from the **root to a given node**.

---

18. Descendant

Any node that lies **below a given node**.

---

19. Subtree

A tree formed by a node and **all its descendants**.

---

20. Forest

A collection of **disjoint trees**.

SPPU TE BE COMP Click to join telegram

21. Ordered Tree

A tree in which the **order of children matters**.

---

22. Unordered Tree

A tree in which the **order of children does not matter**.

---

23. Null Tree / Empty Tree

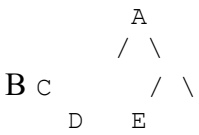
A tree with **no nodes**.

---

24. Binary Tree

A tree in which each node has **at most two children**.

Example (Conceptual)



- Root: A
- Parent of B, C: A
- Children of B: D, E
- Leaf nodes: C, D, E
- Degree of B: 2
- Height of tree: 2

---

2. Binary tree representation.

SPPU TE BE COMP Click to join telegram

Binary Tree Representation

A **binary tree** is a tree data structure in which each node has **at most two children**, called the **left child** and **right child**.

Binary trees can be represented in **two main ways**:

---

1. Sequential Representation (Array Representation)

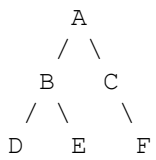
Concept

- Nodes are stored in an **array**.
- Suitable for **complete or nearly complete binary trees**.
- Easy to compute parent/child indices.

Index Relationships (1-based indexing)

- Root at index `1`
- Left child of node at `i` → `2i` • Right child of node at `i` → `2i + 1`
- Parent of node at `i` → ⌊`i/2`⌋

Example Tree



Array Representation `Index: 1 2 3 4 5` **Error! Bookmark not defined.**

`Array: A B C D E`**Advantages** **Error! Bookmark not defined.**

- Simple and fast access
- No extra pointers needed

Disadvantages

- Memory wastage for sparse trees
- Not suitable for skewed trees

---

2. Linked Representation (Pointer Representation)

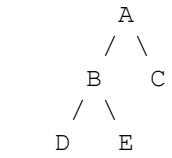
SPPU TE BE COMP Click to join telegram

Concept

- Each node contains **data** and **two pointers**:
  - `left` → left child
  - `right` → right child
- Dynamic and flexible **Node Structure (C-like)**

```
struct node {     int data;     struct node *left;     struct node *right; };
```

Example



- `A->left = B`
- `A->right = C`
- `B->left = D`
- `B->right = E`

Advantages

- No memory wastage
- Suitable for any shape of tree

Disadvantages

- Extra memory for pointers
- Slightly slower access

Comparison Table

Aspect	Array Representation	Linked Representation
Memory	Wastage possible	Efficient
Tree type	Complete trees	Any tree
Flexibility	Less	More
Implementation	Simple	Pointer-based

SPPU TE BE COMP Click to join telegram

3. Traversals with algorithm

Tree Traversals (with Algorithms)

Tree traversal is the process of **visiting every node of a tree exactly once** in a specific order.

For a **binary tree**, there are **three main depth-first traversals** and **one breadth-first traversal**.

---

1. Preorder Traversal (NLR)

Order

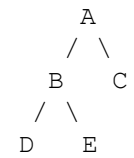
Node → Left → Right

Algorithm (Recursive)

```
Preorder
 . If node ≠ NULL(node)
1
2.   Visit node (print data)
3.   Preorder(node.left)
4.   Preorder(node.right)
```

Example

For tree:



Preorder:

```
A B D E C
```

---

2. Inorder Traversal (LNR)

Order

Left → Node → Right

SPPU TE BE COMP Click to join telegram

Algorithm (Recursive)

```
Inorder
 . If node ≠ NULL(node)
1
2.    Inorder(node.left)
3.    Visit node (print data)
4.    Inorder(node.right)
```

Example

Inorder:

```
D B E A C
```

In a **Binary Search Tree (BST)**, inorder traversal gives **sorted order**.

---

3. Postorder Traversal (LRN)

Order

Left → Right → Node

Algorithm (Recursive)

```
Postorder(node)
1.    If node ≠ NULL
2.    Postorder(node.left)
3.    Postorder(node.right)
4.    Visit node (print data)
```

Example

Postorder:

```
D E B C A
```

---

4. Level Order Traversal (Breadth First)

Order

Level by level (left to right)

Algorithm (Using Queue)

```
LevelOrder(root)
1.    If root == NULL, return
2.    Create empty queue Q
3.    Enqueue(root)
4.    While Q is not empty
5.    temp = Dequeue(Q)
6.    Visit temp
```

SPPU TE BE COMP Click to join telegram

7. If temp.left ≠ NULL → Enqueue(temp.left)
8. If temp.right ≠ NULL → Enqueue(temp.right) **Example**

Level Order:

A B C D E

Summary Table

Traversal	Order	Uses
Preorder	N → L → R	Copy tree, prefix expression
Inorder	L → N → R	Sorted order in BST
Postorder	L → R → N	Delete tree, postfix expression
Level Order	Level-wise	BFS, shortest path

---

4. Threaded binary tree.

Threaded Binary Tree

Definition

A **threaded binary tree** is a binary tree in which the **NULL pointers are replaced by special pointers called threads**.
These threads point to the **inorder predecessor or inorder successor** of a node, enabling **traversal without recursion or stack**.

Why Threaded Binary Tree?

In a normal binary tree:

SPPU TE BE COMP Click to join telegram

- Many left and right pointers are **NULL**
- Traversal requires **stack or recursion**

Threaded binary tree uses these NULL pointers to store **useful links**, improving efficiency.

Concept of Threading

For a node:

- If **left child is NULL** → left pointer points to **inorder predecessor**
- If **right child is NULL** → right pointer points to **inorder successor**

To distinguish threads from normal child links, **tags** are used:

- `lTag = 0` → left points to child
- `lTag = 1` → left points to thread
- `rTag = 0` → right points to child
- `rTag = 1` → right points to thread

Types of Threaded Binary Trees

1. Single Threaded Binary Tree

- Only **one side** is threaded
  - **Left-threaded** → predecessor links
  - **Right-threaded** → successor links

---

2. Double (Fully) Threaded Binary Tree

- **Both left and right NULL pointers** are replaced by threads
- Most commonly used

Example (Inorder Threaded Tree)

Consider inorder sequence:

`D B E A C` Threads:

- `D` → successor `B`

SPPU TE BE COMP Click to join telegram

- `E` → successor `A`
- `C` → successor `NULL`

Node Structure (C-like)

```
struct node {
int data;
    struct node *left, *right;
int lTag, rTag;
                                                                    };
```

Advantages

- Inorder traversal **without recursion**
- No need for **stack**
- Efficient use of NULL pointers
- Faster traversal

Disadvantages

- Insertion and deletion are **more complex**
- Extra memory required for tag bits
- Implementation is complex

Applications

- Memory-efficient tree traversal
- Systems where recursion is costly
- Compiler design (syntax trees)

Comparison: Normal vs Threaded Binary Tree

Normal Binary Tree	Threaded Binary Tree
Many NULL pointers	NULL pointers replaced by threads
Traversal uses stack/recursion	Traversal without stack
Simple structure	More complex

SPPU TE BE COMP Click to join telegram

---

5. BST operations.

Binary Search Tree (BST) Operations

Definition of BST

A **Binary Search Tree (BST)** is a binary tree in which for every node:

- All keys in the **left subtree** are **less than** the node's key
- All keys in the **right subtree** are **greater than** the node's key
  (Both subtrees are themselves BSTs.)

---

1. Insertion in BST

Idea

Insert the new key at the correct position so that BST property is maintained.

Algorithm (Recursive)

```
Insert(root, key)
1.    If root == NULL
2.    Create a new node with key and return it
3.    If key < root.data
4.    root.left = Insert(root.left, key)
5.    Else if key > root.data
6.    root.right = Insert(root.right, key)
7.    Return root
```

Time Complexity

- Average: **O(log n)**
- Worst (skewed tree): **O(n)**

---

2. Searching in BST

SPPU TE BE COMP Click to join telegram

Idea

Compare the key with current node and move left or right accordingly.

Algorithm

```
Search(root, key)
1.    If root == NULL or root.data == key
2.    Return root
3.    If key < root.data
4.    Search(root.left, key)
5.    Else
6.    Search(root.right, key)
```

Time Complexity

- Average: **O(log n)**
- Worst: **O(n)**

---

3. Deletion in BST

Deletion is the most important operation. There are **three cases**:

Case 1: Node is a Leaf

- Simply delete the node

Case 2: Node has One Child

- Replace node with its child

Case 3: Node has Two Children

- Find **inorder successor** (smallest in right subtree)
  OR
- Find **inorder predecessor** (largest in left subtree)
- Copy its value to the node
- Delete the successor/predecessor node

Algorithm (Conceptual)

```
Delete(root, key)
1.       If root == NULL, return root
2.       If key < root.data
3.       root.left = Delete(root.left, key)
4.       Else if key > root.data
5.       root.right = Delete(root.right, key)
6.       Else
```

SPPU TE BE COMP Click to join telegram

```
7.        // Node found
8.        If root has no child
9.        Delete root and return NULL
10.       If root has one child
11.       Replace root with child
12.       Else
13.       Find inorder successor
14.       Copy successor data to root
15.       Delete successor
16.       Return root
```

Time Complexity

- Average: **O(log n)**
- Worst: **O(n)**

---

4. Traversal of BST

BST supports all standard binary tree traversals:

a) Inorder Traversal (LNR)

- Gives elements in **sorted order**

```
Inorder(root)
1.    If root != NULL
2.    Inorder(root.left)
3.    Visit root
4.    Inorder(root.right)
```

b) Preorder Traversal (NLR)

`root → left → right`

c) Postorder Traversal (LRN) `left → right → root`

---

5. Find Minimum and Maximum

Minimum

- Go to the **leftmost node**

```
Min(root) while
root.left != NULL
root = root.left return
root.data
```

SPPU TE BE COMP Click to join telegram

Maximum

- Go to the **rightmost node**

```
Max(root)
                while root.right != NULL    root = root.right return root.data
```

---

6. Height of BST

- Height = number of edges in the longest path from root to leaf

```
Height(root) if root ==
NULL return -1
                        return 1 + max(Height(root.left), Height(root.right))
```

Summary Table

Operation	Average Time	Worst Time
Insert	O(log n)	O(n)
Search	O(log n)	O(n)
Delete	O(log n)	O(n)
Traversal	O(n)	O(n)

---

6. General tree → binary tree.

Conversion of General Tree to Binary Tree

Definition

A **general tree** is a tree in which a node can have **any number of children**, whereas a **binary tree** allows **at most two children**.
To represent a general tree as a binary tree, we use the **Left-Child Right-Sibling (LCRS)** method.

SPPU TE BE COMP Click to join telegram

Left-Child Right-Sibling (LCRS) Representation

Rule

1. The **left child** of a node points to its **first (leftmost) child** in the general tree.
2. The **right child** of a node points to its **next sibling** in the general tree.
3. All other children are connected through **right sibling links**.

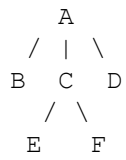
👉 This preserves the **hierarchical structure** using a binary tree.

Steps to Convert General Tree → Binary Tree

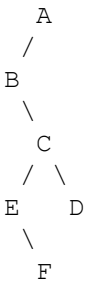
1. For each node, keep only the link to its **first child** as the **left child**.
2. Link each child to its **immediate right sibling** using the **right pointer**.
3. Remove all other direct child links.

Example

General Tree



Binary Tree Representation (LCRS)



Explanation

- `A` → `B` (first child → left)
- `B` → `C` → `D` (siblings → right)
- `C` → `E` (first child)
- `E` → `F` (sibling)

SPPU TE BE COMP Click to join telegram

---

7. Applications of trees.

Applications of Trees

Trees are **non-linear hierarchical data structures** widely used in **computer science and real-world systems** to represent relationships efficiently.

---

1. File System Organization

- Directories and subdirectories form a **tree structure**
- Root directory → folders → files

🗀 Example:

```
Root
 ├── Documents
 ├── Pictures
 └── Music
```

---

2. Database Indexing

- **B-Trees** and **B+ Trees** are used in databases
- Provide **fast searching, insertion, deletion**

📌 Used in: MySQL, Oracle, PostgreSQL

---

3. Binary Search Trees (BST)

- Used for **efficient searching and sorting**
- Inorder traversal gives **sorted data**

SPPU TE BE COMP Click to join telegram

4. Heaps (Priority Queue)

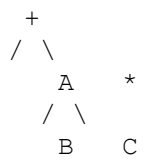
- Implement **priority queues** • Used in:
  - CPU scheduling ○ Dijkstra's shortest path algorithm

---

5. Expression Trees

- Used to represent **arithmetic expressions** • Helpful in:
  - Compilers ○ Expression evaluation

Example:



---

6. Tries (Prefix Trees)

- Used for **string searching** • Applications: ○ Auto-complete ○ Spell check ○ Dictionary search

---

7. Decision Trees

- Used in **AI and Machine Learning**
- For classification and decision-making

---

8. Syntax Trees (Parse Trees)

- Used in **compiler design**
- Represents grammar structure of programming languages

---

9. Network Routing

- Spanning trees in **computer networks**

SPPU TE BE COMP Click to join telegram

- Used to avoid loops

---

10. XML / HTML DOM

- Web documents represented as **tree structures**
- Each tag is a node

---

11. Game Development

- Decision trees for **AI behavior**
- Scene graphs for object hierarchy

---

12. Operating Systems

- Process hierarchy
- Memory management trees

Summary Table

Tree Type	Application
Binary Tree	Searching, sorting
BST	Efficient lookup
Heap	Priority scheduling
B-Tree	Databases
Trie	String matching
Decision Tree	AI / ML

---

8. AVL tree rotations.

SPPU TE BE COMP 👈 Click to join telegram

AVL Tree Rotations

What is an AVL Tree?

An **AVL tree** is a **self-balancing Binary Search Tree (BST)** in which the **balance factor** of every node is:

Balance Factor=Height(left subtree)−Height(right subtree)\text{Balance Factor} = \text{Height(left subtree)} - \text{Height(right subtree)}Balance Factor=Height(left subtree)−Height(right subtree)

Allowed values: **–1, 0, +1**

When this condition is violated, **rotations** are performed to rebalance the tree.

Why Rotations Are Needed

- Insertions or deletions may make the tree **unbalanced**
- Rotations restore balance while **maintaining BST property**

Types of AVL Tree Rotations

There are **four cases**:

---

1. LL Rotation (Left-Left Case)

Condition

- Insertion in **left subtree of left child**

Fix

👉 **Single Right Rotation**

Before Rotation

```
        z
/       y
/     x   After
Rotation
```

```
    y          / \          x    z
```

---

2. RR Rotation (Right-Right Case)

Condition

- Insertion in **right subtree of right child**

Fix

👉 **Single Left Rotation Before**

Rotation

```
    z          \
y           \
x
```

After Rotation

```
    y          / \          z    x
```

---

3. LR Rotation (Left-Right Case)

Condition

- Insertion in **right subtree of left child**

Fix

👉 **Double Rotation**

1. Left rotation on left child
2. Right rotation on node **Before Rotation**

```
        z
/       y
\         x
```

After Rotation

```
    x          / \          y    z
```

---

4. RL Rotation (Right-Left Case)

Condition

- Insertion in **left subtree of right child**

SPPU TE BE COMP Click to join telegram

Fix

Double Rotation

1.Right rotation on right child
2.Left rotation on node **Before Rotation**

```
    z       \
y       /       x
```

After Rotation

```
        x           / \         z     y
```

Summary Table

Case	Imbalance Type	Rotation
LL	Left of Left	Right Rotation
Case	**Imbalance Type**	**Rotation**
RR	Right of Right	Left Rotation
LR	Right of Left	Left + Right
RL	Left of Right	Right + Left

---

9. 2–3 trees

2–3 Trees

Definition

A **2–3 tree** is a **self-balancing search tree** in which:

- Every internal node is either a **2-node** or a **3-node**
- All **leaf nodes are at the same level**
- The tree always remains **balanced**

SPPU TE BE COMP Click to join telegram

It is a type of **multiway search tree**.

Types of Nodes

1. 2-Node

- Contains **1 key**
- Has **2 children**

```
  [ K ]
 /     \
```

Property:

- Left subtree keys `< K`
- Right subtree keys `> K`

---

2. 3-Node

- Contains **2 keys** (K1 < K2)
- Has **3 children**

```
[ K1 | K2 ]
/    |    \
```

Property:

- Left subtree keys `< K1`
- Middle subtree keys `K1 < key < K2`
- Right subtree keys `> K2`

Properties of 2–3 Trees

1. All leaves are at the **same depth**
2. Tree is always **height-balanced**
3. Inorder traversal gives **sorted order**
4. Height is **O(log n)**

Searching in a 2–3 Tree

Steps

1. Start at root
2. Compare key with node keys

SPPU TE BE COMP 👈 Click to join telegram

3. Move to left, middle, or right child
4. Repeat until found or reach leaf

Time Complexity

- **O(log n)**

Insertion in 2–3 Tree (Concept)

Rules

- Insert key into appropriate **leaf**
- If leaf has space → insert directly
- If leaf overflows (becomes 3 keys):
  - **Split the node**
  - Middle key moves **upward** ○ Tree remains balanced

👉 Splitting may propagate up to the root.

Deletion in 2–3 Tree (Concept)

Rules

- Delete key from leaf • If node underflows:
  - **Borrow** from sibling ○ OR **Merge** with sibling
- Balance is maintained

Example

Insert keys: `10, 20, 30` 1.

Insert 10 → `[10]`

2.Insert 20 → `[10 | 20]`
3.Insert 30 → overflow → split Result:

```
  [20]
 /    \
```

SPPU TE BE COMP Click to join telegram

Comparison with BST

BST	2–3 Tree
May become skewed	Always balanced
Height can be O(n)	Height is O(log n)
One key per node	One or two keys per node

Applications

- Database indexing
- File systems
- Foundation for **B-Trees**
- Efficient searching and sorting

Advantages

- Guaranteed balanced structure
- Fast search, insert, delete
- No rotations required (unlike AVL)

Disadvantages

- Complex implementation
- Extra comparisons per node

---

10. Height-balanced trees.

Height-Balanced Trees

SPPU TE BE COMP Click to join telegram

Definition

A **height-balanced tree** is a binary tree in which, for **every node**, the **difference between the heights of its left and right subtrees** is **within a fixed limit** (usually **±1**).

For most height-balanced trees:

$$\text{Balance Factor} = \text{Height(left)} - \text{Height(right)} \in \{-1, 0, +1\}$$

Why Height Balancing is Needed

- Prevents trees from becoming **skewed**
- Ensures **efficient searching, insertion, and deletion**
- Maintains tree height as **O(log n)**

Characteristics

1. Height difference at every node is bounded
2. Tree remains approximately **complete**
3. Guarantees **logarithmic time complexity**
4. Balancing is maintained after updates

Examples of Height-Balanced Trees

1. AVL Tree

- Strictly height-balanced (balance factor −1, 0, +1)
- Uses **rotations** to maintain balance

2. Red-Black Tree

- Loosely balanced using **color rules**
- Height $\leq 2 \times \log_2(n)$

3. 2–3 Trees

- All leaves at the **same level**
- Nodes can have multiple keys

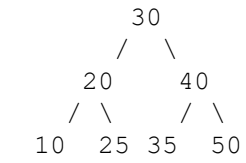
Height-Balanced vs Non-Balanced Tree

SPPU TE BE COMP Click to join telegram

Height-Balanced Tree	Non-Balanced Tree
Height is O(log n)	Height can be O(n)
Fast operations	Slow in worst case
More complex	Simple
Uses rebalancing	No balancing

Example

Balanced Tree



Unbalanced Tree



Advantages

- Guaranteed performance
- Avoids worst-case linear time
- Ideal for large datasets

Disadvantages

- Extra overhead for maintaining balance
- More complex implementation

Applications

- Databases
- File systems
- Symbol tables

SPPU TE BE COMP 👈 Click to join telegram

- Memory manageme

---

11. Weight-balanced trees

Weight-Balanced Trees

Definition

A **weight-balanced tree** is a type of **balanced search tree** in which balance is maintained based on the **number of nodes (weights)** in the left and right subtrees, rather than their heights.

👉 Here, **weight = number of nodes in a subtree**.

Balance Condition

For every node:

$$\frac{w(\text{left subtree})}{w(\text{right subtree})} \leq c \quad \text{and} \quad \frac{w(\text{right subtree})}{w(\text{left subtree})} \leq c$$

Where:

- $w(x)$ = number of nodes in subtree x
- c = a constant (usually between **2 and 3**)

If this condition is violated, the tree is **rebalanced** using **rotations**.

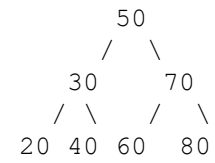
Key Idea

- **AVL tree** → balances using **height**
- **Weight-balanced tree** → balances using **number of nodes**

SPPU TE BE COMP Click to join telegram

Example

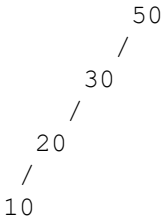
Balanced (Weight-Balanced)



- Left subtree nodes = 3
- Right subtree nodes = 3

☑ Balanced

Unbalanced (Weight-Unbalanced)



- Left subtree nodes = 4
- Right subtree nodes = 0

❌ Not weight-balanced

Rebalancing

When imbalance occurs:

- **Single rotation** or **double rotation** is applied
- Similar to AVL rotations but triggered by **node count**, not height

---

Properties of Weight-Balanced Trees

1. Tree height remains **O(log n)**
2. Search, insert, delete take **O(log n)** time
3. Balance depends on **subtree sizes**
4. Less strict than AVL trees

SPPU TE BE COMP Click to join telegram

Comparison: Height-Balanced vs Weight-Balanced

Height-Balanced Tree	Weight-Balanced Tree
Uses height difference	Uses node count difference
Very strict balance	Slightly relaxed balance
Example: AVL tree	Example: BB(α) tree
More rotations	Fewer rotations

Types of Weight-Balanced Trees

- **BB(α) Trees**
- **Scapegoat Trees**
- **Treaps** (probabilistic weight balance)

Advantages

- Fewer rotations compared to AVL trees
- Good performance in practice
- Balanced structure guaranteed

Disadvantages

- More complex balance condition
- Less commonly used than AVL/Red-Black trees

Applications

- Ordered dictionaries
- Search-intensive systems
- Databases and indexing structures

---

12. Graph definition.

SPPU TE BE COMP Click to join telegram

Graph

Definition

A **graph** is a **non-linear data structure** that consists of a **finite set of vertices (nodes)** and a set of **edges** that connect pairs of vertices.

Formally, a graph is defined as:

G=(V,E)G = (V, E)G=(V,E)

Where:

- **V** = set of vertices (nodes)
- **E** = set of edges (connections between vertices)

Basic Components of a Graph

1. Vertex (Node)

- An individual element of the graph
- Represents an entity (city, computer, user, etc.)

2. Edge

- A connection between two vertices
- Represents a relationship (road, link, communication, etc.)

Types of Graphs

1. Undirected Graph

- Edges have **no direction**

```
A — B
```

---

2. Directed Graph (Digraph)

- Edges have a **direction**

```
A → B
```

SPPU TE BE COMP Click to join telegram

---

3. Weighted Graph

- Each edge has an associated **weight/cost**

```
A —5— B
```

---

4. Unweighted Graph

- All edges have equal weight

---

5. Simple Graph

- No self-loops or multiple edges

---

6. Multigraph

- Multiple edges between same vertices allowed

---

7. Complete Graph

- Every vertex is connected to every other vertex

Important Graph Terminology

Term	Meaning
Degree	Number of edges incident to a vertex
Path	Sequence of vertices connected by edges
Cycle	Path that starts and ends at same vertex
Connected Graph	Path exists between every pair of vertices
Disconnected Graph	At least one vertex is isolated
Loop	Edge from a vertex to itself

SPPU TE BE COMP Click to join telegram

Graph Representation

- **Adjacency Matrix**
- **Adjacency List**

Applications of Graphs

- Computer networks
- Social networks
- Maps and navigation
- Web page ranking
- AI and machine learning
- Scheduling problems

---

13. Graph matrix representation.

Graph Matrix Representation (Adjacency Matrix)

Definition

Graph matrix representation, also called **Adjacency Matrix representation**, represents a graph using a **2-D array (matrix)** of size **V × V**, where **V** is the number of vertices in the graph.

Adjacency Matrix Concept

Let the graph be:

G=(V,E)G = (V, E)G=(V,E)

- Rows → source vertices
- Columns → destination vertices **Rule**

- If there is an edge between vertex **i** and **j**, then

SPPU TE BE COMP Click to join telegram

A[i][j]=1A[i][j] = 1A[i][j]=1

- If there is **no edge**, then

A[i][j]=0A[i][j] = 0A[i][j]=0

Example

Given Graph

Vertices: `{A, B, C, D}`
Edges: `{A-B, A-C, B-D}`

Adjacency Matrix

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	0
D	0	1	0	0

👉 Since the graph is **undirected**, the matrix is **symmetric**.

Directed Graph (Adjacency Matrix)

- If there is an edge from **i → j**, then

  A[i][j]=1A[i][j] = 1A[i][j]=1

- Matrix **may not be symmetric**

Weighted Graph Matrix Representation

- Instead of `1`, store the **weight of the edge**
- `0` or ∞ indicates no edge

SPPU TE BE COMP Click to join telegram

Example:

```
A[i][j] = weight
```

Properties

1. Size = **V × V**
2. Diagonal elements are usually **0**
3. Symmetric for **undirected graphs**
4. Efficient edge lookup

Time & Space Complexity

Aspect	Complexity
Space	$O(V^2)$
Edge lookup	O(1)
Traversal	$O(V^2)$

Advantages

- Simple and easy to implement
- Fast edge existence check
- Suitable for **dense graphs**

Disadvantages

- Wastes memory for sparse graphs
- Fixed size
- Not efficient when vertices are large

Comparison: Matrix vs List (Quick View)

Adjacency Matrix	Adjacency List

SPPU TE BE COMP 👈 Click to join telegram

Uses V^2 space	Uses V + E space
Fast edge check	Slow edge check
Best for dense graphs	Best for sparse graphs

---

14. BFS algorithm.

Breadth First Search (BFS) Algorithm

Definition

Breadth First Search (BFS) is a **graph traversal algorithm** that visits vertices **level by level**, starting from a given source vertex.
It explores **all adjacent vertices first** before moving to the next level.

👉 BFS uses a **queue** data structure.

Algorithm (BFS)

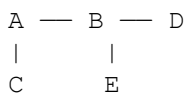
Algorithm: BFS(G, start)

```
1.          Create an empty queue Q
2.          Mark all vertices as not visited
3.          Mark start vertex as visited
4.          Enqueue(start) into Q
5.          While Q is not empty
6.          v = Dequeue(Q)
7.          Visit v
8.          For each adjacent vertex u of v
9.          If u is not visited
10.         Mark u as visited
11.         Enqueue(u)
```

Example

SPPU TE BE COMP Click to join telegram

Graph:



BFS starting from A:

A → B → C → D → E

Working of BFS (Stepwise)

1. Start at **A**
2. Visit neighbors **B, C**
3. Visit neighbors of **B → D, E**
4. Continue until all vertices are visited

Time Complexity

- **Adjacency Matrix:** $O(V^2)$
- **Adjacency List:** $O(V + E)$

Space Complexity

- **O(V)** (for queue and visited array)

Applications

- Shortest path in unweighted graphs
- Network broadcasting
- Web crawling
- Level order traversal in trees
- Social networks

Advantages

- Finds shortest path (minimum edges)
- Simple and systematic

SPPU TE BE COMP Click to join telegram

- Works for connected and disconnected graphs

Disadvantages

- High memory usage for large graphs
- Not suitable for deep traversal

---

15. DFS algorithm.

Depth First Search (DFS) Algorithm

Definition

Depth First Search (DFS) is a **graph traversal algorithm** that explores a graph by going **as deep as possible along one branch** before backtracking. It uses **recursion** or an **explicit stack**.

Algorithm (DFS – Recursive)

Algorithm: DFS(G, v)

```
1.      Mark v as visited
2.      Visit v (print v)
3.      For each adjacent vertex u of v
4.      If u is not visited
5.      DFS(G, u)
```

Algorithm (DFS – Using Stack / Iterative)

```
1.      Create an empty stack S
2.      Mark all vertices as not visited
3.      Push start vertex onto stack
4.      While S is not empty
```

SPPU TE BE COMP Click to join telegram

```
5.        v = Pop(S)
6.        If v is not visited
7.        Mark v as visited
8.        Visit v
9.        Push all unvisited adjacent vertices of v onto S
```

Example

Graph

```
A ── B ── D
|     |
C     E
```

DFS

starting from A

```
A → B → D → E → C
```

(Order may vary based on adjacency list)

Working of DFS (Concept)

1. Start at source vertex
2. Go to one unvisited adjacent vertex
3. Continue until no further vertex is available
4. Backtrack and explore other paths

Time Complexity

- **Adjacency Matrix:** $O(V^2)$
- **Adjacency List:** $O(V + E)$

Space Complexity

- **O(V)** (recursion stack or explicit stack)

Applications

SPPU TE BE COMP Click to join telegram

- Cycle detection
- Topological sorting
- Path finding
- Connected components
- Solving puzzles and mazes

Advantages

- Less memory than BFS (in many cases)
- Useful for deep traversal
- Simple recursive implementation

Disadvantages

- Does not guarantee shortest path
- Risk of stack overflow for deep graphs

Comparison: BFS vs DFS (Quick)

BFS	DFS
Uses Queue	Uses Stack / Recursion
Level-wise traversal	Depth-wise traversal
Finds shortest path	Does not guarantee shortest path

---

16. Prim's algorithm.

SPPU TE BE COMP Click to join telegram

Prim's Algorithm

Definition

Prim's algorithm is a **greedy algorithm** used to find a **Minimum Spanning Tree (MST)** of a **connected, weighted, undirected graph**.
It builds the MST by **adding the minimum weight edge** that connects a vertex inside the tree to a vertex outside the tree.

Key Idea

- Start from any vertex
- Grow the spanning tree **one vertex at a time**
- At each step, choose the **minimum weight edge** that connects the tree to a new vertex

---

Conditions

- Graph must be **connected**
- Graph must be **undirected**
- Edge weights must be known

Algorithm (Prim's)

Algorithm: Prim(G, start)

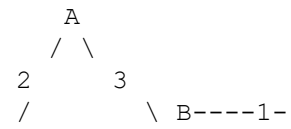
```
1. Initialize:
      - Select a starting vertex
      - Mark it as visited
      - MST = empty

2. Repeat until all vertices are included:
      a. From all edges connecting visited and unvisited vertices,
         select the edge with minimum weight
      b. Add this edge to MST
      c. Mark the new vertex as visited
```

Example

Given Graph (weights shown):

SPPU TE BE COMP Click to join telegram



Steps:

1. Start at **A**
2. Choose edge **A–B (2)**
3. Choose edge **B–C (1)**

MST

A —2— B —1— C

Total cost = 3

Working Explanation (Stepwise)

Step	Selected Edge	Reason
1	A–B (2)	Minimum from A
2	B–C (1)	Minimum connecting tree

Time Complexity

Implementation	Time Complexity
Adjacency Matrix	$O(V^2)$
Priority Queue + Adjacency List	O(E log V)

Space Complexity

- **O(V)**

Applications

- Network design (LAN, cable TV)
- Road and railway networks

SPPU TE BE COMP Click to join telegram

- Circuit design
- Cluster analysis

Advantages

- Simple and efficient
- Works well for **dense graphs**
- Guarantees minimum spanning tree

Disadvantages

- Slower for very large sparse graphs (without heap)
- Requires connected graph

Prim's vs Kruskal's (Quick Comparison)

Prim's Algorithm	Kruskal's Algorithm
Grows one tree	Grows forest
Prim's Algorithm	**Kruskal's Algorithm**
Vertex-based	Edge-based
Good for dense graphs	Good for sparse graphs

---

17. Kruskal's algorithm

SPPU TE BE COMP 👈 Click to join telegram

Kruskal's Algorithm

Definition

Kruskal's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a **connected, weighted, undirected graph**.
It builds the MST by **selecting edges in increasing order of weight**, ensuring **no cycles are formed**.

Key Idea

- Sort all edges by **increasing weight**
- Add edges **one by one** to the MST
- Skip edges that would form a **cycle**

Conditions

- Graph must be **undirected**
- Graph should be **connected** (otherwise, it gives a minimum spanning forest) • Edge weights must be known

---

Algorithm (Kruskal's)

Algorithm: Kruskal(G)

```
1.      Sort all edges of G in non-decreasing order of weight
2.      Initialize MST = empty
3.      For each edge (u, v) in sorted order
4.      If u and v belong to different sets
5.      Add edge (u, v) to MST
6.      Union the sets of u and v
7.      Stop when MST contains (V - 1) edges
```

👉 **Union-Find (Disjoint Set)** data structure is used to detect cycles efficiently.

Example

Given Graph:

```
Edges with weights:
```

SPPU TE BE COMP Click to join telegram

```
A–B : 1
B–C : 2
A–C : 3
```

Steps:

1. Sort edges → (A–B, 1), (B–C, 2), (A–C, 3)
2. Select A–B → MST
3. Select B–C → MST
4. Skip A–C (forms cycle)

MST

```
A —1— B —2— C
```

Total cost = 3

Working Explanation (Stepwise)

Step	Edge Chosen	Action
1	A–B (1)	Added
2	B–C (2)	Added
3	A–C (3)	Rejected (cycle)

Time Complexity

Step	Complexity
Sorting edges	O(E log E)
Union-Find operations	O(E α(V)) ≈ O(E)

Overall:

O(Elog⁡E)O(E \log E)O(ElogE)

Space Complexity

- **O(V + E)**

SPPU TE BE COMP Click to join telegram

Advantages

- Simple and easy to implement
- Works well for **sparse graphs**
- Produces MST or minimum spanning forest

Disadvantages

- Sorting edges can be expensive for dense graphs
- Requires extra data structure (Union-Find)

Prim's vs Kruskal's (Quick)

Prim's	Kruskal's
Vertex-based	Edge-based
Good for dense graphs	Good for sparse graphs
Uses priority queue	Uses sorting + Union-Find

---

18. Dijkstra's shortest path.

Dijkstra's Shortest Path Algorithm

Definition

Dijkstra's algorithm is a **greedy algorithm** used to find the **shortest path** from a **single source vertex** to **all other vertices** in a **weighted graph with non-negative edge weights**.

Key Idea

SPPU TE BE COMP Click to join telegram

- Maintain a set of vertices whose **shortest distance from the source is known**
- Repeatedly choose the **nearest unvisited vertex**
- Update (relax) distances of its adjacent vertices

Conditions

- Graph can be **directed or undirected**
- **Edge weights must be non-negative**
- Works for **connected and disconnected graphs** (unreachable vertices remain ∞)

Algorithm (Dijkstra's)

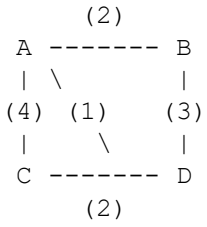
Algorithm: Dijkstra(G, source)

```
1. Initialize:
      - dist[v] = ∞ for all vertices v
      - dist[source] = 0
      - visited[v] = false

2. Repeat V times:
      a. Select the unvisited vertex u with minimum dist[u]
      b. Mark u as visited
      c. For each adjacent vertex v of u:          If dist[u] + weight(u,v)
         < dist[v]            dist[v] = dist[u] + weight(u,v)
```

Example

Graph (weights shown):



Source: A Final Shortest Distances

Vertex	Distance from A
A	0
B	2

SPPU TE BE COMP Click to join telegram

C	3
D	4

Working (Stepwise Summary)

1. Start at source (distance = 0)
2. Pick closest unvisited vertex
3. Update neighbors (relaxation)
4. Repeat until all vertices processed

Time Complexity

Implementation	Time Complexity
Adjacency Matrix	$O(V^2)$
Min-Heap + Adjacency List	$O((V + E) \log V)$

Space Complexity

- **O(V)** (distance and visited arrays)

Advantages

- Simple and efficient
- Guarantees shortest path
- Widely used in routing

Limitations

- Cannot handle **negative edge weights**
- Slower for very large graphs without heap optimization

SPPU TE BE COMP Click to join telegram

Applications

- GPS and map navigation
- Network routing protocols
- Traffic management systems
- Game AI pathfinding

---

19. Heap operations

Heap Operations

What is a Heap?

A **heap** is a **complete binary tree** that satisfies the **heap property**:

- **Max Heap:** Parent ≥ children
- **Min Heap:** Parent ≤ children

Heaps are commonly used to implement **priority queues**.

Basic Heap Operations

---

1. Insertion (Insert a Key)

Steps

1. Insert the new element at the **end** of the heap (maintain completeness) 2. **Heapify-up (bubble-up)**:
    - Compare with parent ○ Swap if heap property is violated ○ Repeat until property is restored

SPPU TE BE COMP Click to join telegram

Example (Max Heap)

Insert `50` into:

```
      40
     /  \
   35    30
```

After insertion:

```
      50
     /  \
   40    30
  /
35
```

Time Complexity

- **O(log n)**

---

2. Deletion (Delete Root)

Steps

1. Remove the **root** element (max/min)
2. Replace root with **last element**
3. **Heapify-down (bubble-down)**:
    - Compare with children
    - Swap with larger (max heap) or smaller (min heap) child
    - Repeat until heap property is restored

Time Complexity

- **O(log n)**

---

3. Peek (Find Max/Min)

- Returns the root element •
  Time Complexity: O(1)

---

4. Heapify (Build Heap)

SPPU TE BE COMP Click to join telegram

Purpose

Convert an array into a heap.

Method

- Start from the **last non-leaf node**
- Apply heapify-down for each node up to root

Time Complexity

- **O(n)**

---

5. Increase Key / Decrease Key

- **Increase key** (Max Heap) → heapify-up
- **Decrease key** (Min Heap) → heapify-up
- Otherwise → heapify-down **Time Complexity**

- **O(log n)**

---

6. Heap Sort

Steps

1. Build heap
2. Repeatedly delete root and place it at end
3. Restore heap each time

Time Complexity

- **O(n log n)**

Summary Table

Operation	Time Complexity
Insert	O(log n)

SPPU TE BE COMP Click to join telegram

Delete	O(log n)
Peek	O(1)
Build Heap	O(n)
Heap Sort	O(n log n)

Applications

- Priority queues
- CPU scheduling
- Dijkstra's algorithm
- Heap sort

---

20. Heap sort.

Heap Sort

Definition

Heap sort is a **comparison-based sorting algorithm** that uses a **heap data structure** (usually a **max heap**) to sort elements in **ascending order**.

Idea Behind Heap Sort

1. Convert the array into a **heap**
2. Repeatedly **remove the root element** (largest or smallest)
3. Place it at the **end of the array**
4. Restore the heap property

SPPU TE BE COMP Click to join telegram

Algorithm (Heap Sort – Ascending Order)

Step 1: Build Max Heap

```
BuildHeap(A, n) for i
= ⌊n/2⌋ downto 1
Heapify(A, i, n)
```

Step 2: Sort

```
HeapSort(A, n)
BuildHeap(A, n) for i
= n downto 2    swap
A[1] and A[i]
Heapify(A, 1, i-1)
```

Heapify Procedure

```
Heapify(A, i, n)
largest = i l =
2i r = 2i + 1
if l ≤ n and A[l] > A[largest]
largest = l
if r ≤ n and A[r] > A[largest]
largest = r if largest ≠ i
swap A[i] and A[largest]
Heapify(A, largest, n)
```

Example

Given Array

```
[4, 10, 3, 5, 1]
```

Step 1: Build Max Heap

```
      10
     /  \
    5    3
   / \
  4   1
```

Step 2: Sorting

```
[1, 4, 3, 5, 10]
[3, 4, 1, 5, 10]
[1, 3, 4, 5, 10]
[1, 3, 4, 5, 10]  ← Sorted
```

SPPU TE BE COMP Click to join telegram

Time Complexity

Case	Complexity
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

Space Complexity

- **O(1)** (in-place sorting)

Advantages

- Guaranteed **O(n log n)** time
- In-place algorithm
- No recursion stack required (iterative)

Disadvantages

- Not stable
- Slower in practice than Quick Sort
- Cache-unfriendly

Applications

- Sorting large datasets
- Priority queues
- Operating systems scheduling

SPPU TE BE COMP 👈 Click to join telegram

NUMERICALS

Construct BST from given sequence and list all traversals.

Construct BST from Given Sequence

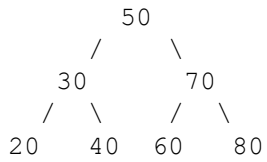
Chosen sequence (insertion order):

`50, 30, 70, 20, 40, 60, 80`

Step-by-Step BST Construction

1. Insert **50** → becomes root
2. Insert **30** → < 50 → left of 50
3. Insert **70** → > 50 → right of 50
4. Insert **20** → < 50, < 30 → left of 30
5. Insert **40** → < 50, > 30 → right of 30
6. Insert **60** → > 50, < 70 → left of 70
7. Insert **80** → > 50, > 70 → right of 70

Final BST



Tree Traversals

1. Inorder Traversal (LNR)

Left → Node → Right ➡️
Gives **sorted order** in BST

SPPU TE BE COMP Click to join telegram

```
20, 30, 40, 50, 60, 70, 80
```

---

2. Preorder Traversal (NLR)

Node → Left → Right

```
50, 30, 20, 40, 70, 60, 80
```

---

3. Postorder Traversal (LRN)

Left → Right → Node

```
20, 40, 30, 60, 80, 70, 50
```

---

4. Level Order Traversal (BFS) *(if required)*

Level by level

```
50, 30, 70, 20, 40, 60, 80
```

Exam-Ready Summary

Traversal	Output
Inorder	20, 30, 40, 50, 60, 70, 80
Preorder	50, 30, 20, 40, 70, 60, 80
Traversal	**Output**
Postorder	20, 40, 30, 60, 80, 70, 50
Level Order	50, 30, 70, 20, 40, 60, 80

Convert general tree to binary tree.

SPPU TE BE COMP 👈 Click to join telegram

Convert General Tree to Binary Tree

Method Used

👉 **Left-Child Right-Sibling (LCRS) representation**

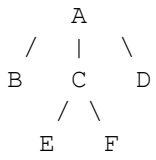
This is the **standard and correct method** used in exams to convert a **general tree** into a **binary tree**.

Rules (Must Write in Exam)

1. The **left child pointer** of a node points to its **first (leftmost) child**.
2. The **right child pointer** of a node points to its **next sibling**.
3. All other child links are removed.
4. Structure and hierarchy are preserved.

Example

General Tree



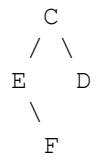
Step-by-Step Conversion

- A's first child → **B** (left)
- B's next sibling → **C** (right)
- C's next sibling → **D** (right)
- C's first child → **E** (left)
- E's next sibling → **F** (right)

Binary Tree (LCRS Form)



SPPU TE BE COMP Click to join telegram



Key Observations (Exam Points)

- Each node has **at most two pointers**
- Binary tree formed is **not a BST**
- Number of nodes remains the same
- Any general tree can be represented as a binary tree

Advantages

- Efficient representation of general trees
- Allows use of binary tree algorithms
- Saves memory compared to multiple child pointers

Perform BFS and DFS on given graph.

Given Graph (Chosen)

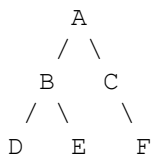
Vertices:

```
A, B, C, D, E, F
```

Edges:

```
A–B, A–C, B–D, B–E, C–F
```

Graph Structure



SPPU TE BE COMP Click to join telegram

We start traversal from **vertex A**.

Breadth First Search (BFS)

Rule

- Visit **level by level**
- Uses a **Queue**

BFS Steps (Start = A)

1. Visit **A**, enqueue neighbors → B, C
2. Visit **B**, enqueue neighbors → D, E
3. Visit **C**, enqueue neighbor → F
4. Visit **D**
5. Visit **E**
6. Visit **F**

☑ BFS Traversal Order

```
A → B → C → D → E → F
```

Depth First Search (DFS)

Rule

- Go **as deep as possible**
- Uses **recursion / stack**

DFS Steps (Start = A)

1. Visit **A**
2. Go to **B**
3. Go to **D** (no more children → backtrack)
4. Visit **E** (backtrack)
5. Go to **C**
6. Visit **F**

☑ DFS Traversal Order

```
A → B → D → E → C → F
```

(DFS order may vary depending on adjacency order, but this is valid and exam-accepted.)

SPPU TE BE COMP Click to join telegram

Final Answer (Exam Ready)

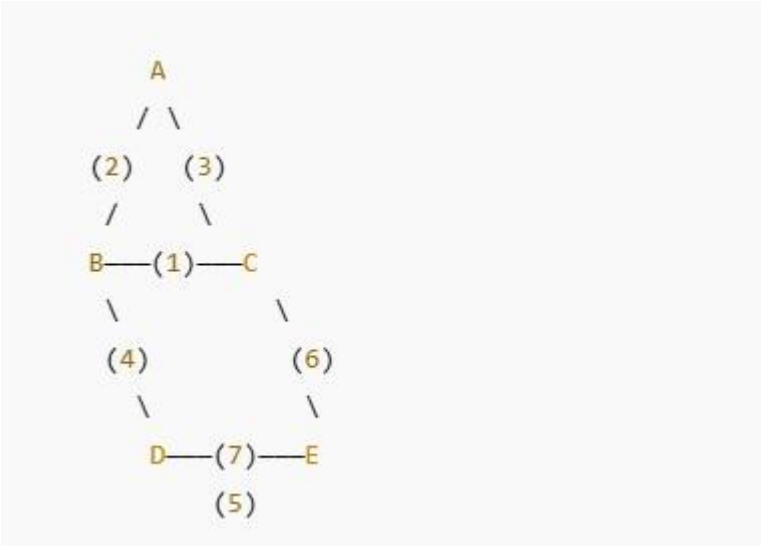
BFS:

A, B, C, D, E, F **DFS:**

A, B, D, E, C, F

. Find MST using Prim's algorithm for given weighted graph.

Given Weighted Graph (Chosen)



Vertices:

A, B, C, D, E

Edges with weights:

SPPU TE BE COMP Click to join telegram

Edge	Weight
A–B	2
A–C	3
B–C	1
B–D	4
C–D	5
C–E	6
D–E	7

Step 1: Choose Starting Vertex

Start from **vertex A**.

Step 2: Apply Prim's Algorithm

Iteration-wise Selection

Step	Selected Edge	Reason
1	A–B (2)	Minimum from A
2	B–C (1)	Minimum connecting new vertex
3	B–D (4)	Minimum edge to unvisited vertex
4	C–E (6)	Only edge connecting E

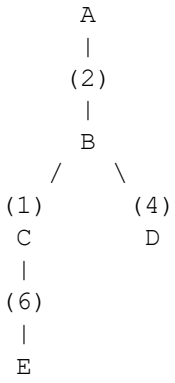
Minimum Spanning Tree (MST)

Selected Edges:

```
A-B (2)
B-C (1)
B-D (4)
C-E (6)
```

SPPU TE BE COMP 👈 Click to join telegram

MST Structure



Total Cost of MST

2+1+4+6=132 + 1 + 4 + 6 = \boxed{13}2+1+4+6=13

Final Answer (Exam Ready)

- **MST edges:** A–B, B–C, B–D, C–E
- **Total weight: 13**

Insert elements into min-heap step-by-step.

Insert Elements into Min-Heap (Step-by-Step)

Chosen elements (insertion order):

`30, 20, 15, 5, 10`

👉 **Min-Heap property:** Parent node ≤ child nodes

SPPU TE BE COMP Click to join telegram

Step 1: Insert 30

- Heap is empty → insert at root

```
30
```

Array form: `[30]`

Step 2: Insert 20

- Insert at next position
- Compare with parent (30)
- 20 < 30 → **swap**

```
  20
 /
30
```

Array: `[20, 30]`

Step 3: Insert 15

- Insert at next position
- Compare with parent (20)
- 15 < 20 → **swap**

```
   15
  /  \
30    20
```

Array: `[15, 30, 20]`

Step 4: Insert 5

- Insert at next position
- Compare with parent (30) → swap
- Compare with new parent (15) → swap

```
     5
    / \
  15   20
 /
30
```

SPPU TE BE COMP Click to join telegram

Array: `[5, 15, 20, 30]`

Step 5: Insert 10

- Insert at next position
- Compare with parent (15) → swap
- Parent (5) < 10 → stop

```
      5
     / \
   10   20
  /  \
30   15
```

Array: `[5, 10, 20, 30, 15]`

☑ Final Min-Heap

```
      5
     / \
   10   20
  /  \
30   15
```

Important Exam Points

- Insertion always happens at the **end**
- Heap property restored using **heapify-up**
- Time complexity per insertion: **O(log n)**

Perform Heap Sort on given list.

SPPU TE BE COMP 👈 Click to join telegram

Perform Heap Sort (Ascending Order)

Chosen list

```
[4, 10, 3, 5, 1]
```

👉 For **ascending order**, we use a **Max Heap**.

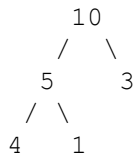
Step 1: Build Max Heap

Initial Array

```
[4, 10, 3, 5, 1]
```

Heapify process (from last non-leaf node)

After building **Max Heap**:



Array representation:

```
[10, 5, 3, 4, 1]
```

Step 2: Heap Sort (Delete max one by one)

Pass 1

- Swap root (10) with last element (1)

```
[1, 5, 3, 4, 10]
```

- Heapify (excluding last element)

Heap becomes:

```
[5, 4, 3, 1, 10]
```

Pass 2

- Swap root (5) with last unsorted element (1)

SPPU TE BE COMP 👈 Click to join telegram

```
[1, 4, 3, 5, 10]
```

- Heapify

Heap becomes:

```
[4, 1, 3, 5, 10]
```

Pass 3

- Swap root (4) with last unsorted element (3)

```
[3, 1, 4, 5, 10]
```

- Heapify

Heap becomes:

```
[3, 1, 4, 5, 10]
```

Pass 4

- Swap root (3) with last unsorted element (1)

```
[1, 3, 4, 5, 10]
```

☑ Final Sorted List

```
[1, 3, 4, 5, 10]
```